

Automated Speech-to-Video ASL Translator

Team 6- Ikshana

19-06-2024

INDEX

- 1) Team Introduction
- 2) What are we Building
- 3) Abstract
- 4) Most Important Use Case
- 5) Business Use Cases
- 6) Project Outline
- 7) Concepts Used
- 8) Data Collection and Processing
- 9) Algorithm Used
- 10) Models used
- 11) Challenges Faced
- 12) Our Learnings
- 13) Future Enhancements
- 14) Deployment Screenshot
- 15) Deployment Metrics

OUR TEAM



THOTA SRIRAM NAIDU
Project Coordinator and R&D



MAYANK KUMAR
Model Development and System
Integration



BHAVADHARINI SHANKAR
Dataset management and research



GURU CHARAN VARMA DOMMARAJU
Model optimizer



HARSH S MENON
R&D and Model Development

Problem Statement

The Automated Speech-to-Video ASL Translator addresses the critical problem of communication barriers between hearing individuals and the deaf or hard of hearing. The project aims to convert spoken language into real-time American Sign Language (ASL) videos, enabling seamless and effective interaction by translating spoken words into comprehensible ASL gestures. It will also help them to fully understand sentiment behind text, ensuring they can fully participate in conversations.



Abstract

Develop a system that converts spoken language into American Sign Language (ASL) videos for effective communication with the deaf and hard-of-hearing community.

Utilizes deep learning methods, specifically Recurrent Neural Networks (RNN), for interpreting speech and translating it into text.

The system generates corresponding ASL gestures from the translated text.

Combines individual ASL gesture videos to form coherent sentences.

Ensures users can understand the translation process, enhancing trust in the system's accuracy.

Allows users to identify and correct translation errors, promoting reliability in communication.

Aims to improve accessibility and communication for the deaf and hard-of-hearing community through accurate and transparent translation.

Important Use Case

The most important use case for an Automated Speech-to-Video ASL Translator is to facilitate real-time communication between hearing individuals and the deaf or hard-of-hearing community in settings such as customer service, healthcare, and public services. This technology allows for instantaneous translation of spoken language into visual ASL, enabling deaf or hard-of-hearing individuals to access information, express needs, and participate in conversations without the need for a live interpreter. This promotes inclusivity, accessibility, and equal opportunity for meaningful interaction in daily life and critical situations.

Business Use Case



1. Education

Facilitates inclusive learning by translating spoken content into ASL videos for deaf students, ensuring they receive the same information as their peers.

2. Entertainment

Provides accessibility in media by translating spoken dialogue into ASL, allowing deaf individuals to enjoy movies, shows, and other content.

3. Workplace Inclusion

Enables deaf employees to participate in meetings and training by translating spoken language into ASL, fostering a more inclusive work environment.

4. Travel and Hospitality

Enhances the experience of deaf travelers by translating spoken announcements and service information into ASL, ensuring they can navigate and enjoy their travels independently.

5. Customer Care

Enhances communication with deaf customers by providing real-time ASL translations for better service accessibility.

6. Announcements

Converts spoken announcements into ASL videos for public places, ensuring deaf individuals receive important information promptly.

Project Outline



Research and Planning

Define the project scope, understand existing technologies, and identify key requirements.

Speech Recognition and NLP

Implement and fine-tune a speech recognition system to accurately transcribe spoken words. Develop an NLP model to detect key phrases that trigger gestures.

Mapping and Database Creation

Create a database of gestures mapped to specific phrases. Integrate the speech and NLP components with the gesture control system to perform gestures.

User Interface & Experience

Create a user-friendly interface for managing the system and customizing gestures.

Testing and Deployment

Ensure the system works reliably in real-world scenarios and meets user needs. Deploy the system for use in various settings and gather user feedback for further improvement.

Concepts Used

STEP-1

Speech Recognition
transcribes spoken words
into text.

STEP-2

—————→ **NLP** processes the text to
detect key phrases.

STEP-3

—————→ **Deep Learning** enhances
the accuracy and
performance of the
system.

STEP-4

**Gesture Execution and
Control** performs the
corresponding gestures.

STEP-5

—————→ **HCI Principles** ensure the
system is user-friendly and
effective.

STEP-6

—————→ **Data Collection and
Processing** provides the
necessary data to train and
refine the system.

Data collecting and processing

Data Collection

Effective data collection is crucial for training the speech recognition, NLP, and gesture recognition components of the system. The steps involved include:

a. Speech Data Collection

Gather diverse and high-quality audio samples to train the speech recognition model.

b. Gesture Data Collection

Gather a comprehensive set of gestures that correspond to key phrases in the speech data.

c. Annotation

Label the collected speech and gesture data with relevant information.



Data Processing

Once the data is collected and annotated, it must be preprocessed and prepared for training the models. This includes:

a. Audio Preprocessing

Prepare audio data for speech recognition.

b. Text Preprocessing

Prepare transcriptions for NLP processing.

c. Gesture Preprocessing

Prepare gesture data for integration and training.



Model Training

Using the processed data, train the models for speech recognition, NLP, and gesture control:

- a. Speech Recognition Model
- b. NLP Model
- c. Gesture Control Model

Integration and Testing

Integrate the trained models into the overall system and test them in real-world scenarios:

a. System Integration

Combine speech recognition, NLP, and gesture control into a cohesive system.

b. User Testing

Test the system with real users to gather feedback and identify areas for improvement.

Algorithm Used

1. Define the CTC Loss Function

- Compute CTC loss between true labels and predicted probabilities.

2. Build the Model

- Construct a neural network with spectrogram inputs to predict probability distributions.

3. Define Prediction Decoding Function

- Decode predicted probabilities into text sequences using CTC decoding.

4. Define Evaluation Callback

- Implement a callback to evaluate and log model performance after each training epoch.

5. Putting It All Together

- Compile the model with CTC loss.
- Train with the custom evaluation callback.

MODELS USED

1. Two Conv2D layers with batch normalization
2. Five Bidirectional GRU layers (each with 256 units)

CHALLENGES FACED

1. Due to certain limitation, google colab did not support Real-time processing. Real-time audio-to-text conversion was a complex task to solve.
2. Testing the system thoroughly to ensure it works well in different real-world situations and obtaining good datasets that cover a wide range of words, sentence patterns, and contexts for both speech and American Sign Language (ASL) are big challenges when developing an Automated Speech-to-Video ASL Translator. Acquiring large, high-quality datasets that include a broad vocabulary and various sentence structures is crucial for training.

OUR LEARNINGS

1. Speech-to-text Conversion with RNN
2. Translation from Text to ASL using NLP
3. Deep Learning for Gesture Generation
4. Video Stitching and Sequencing
5. Real Time Processing

FUTURE ENHANCEMENTS



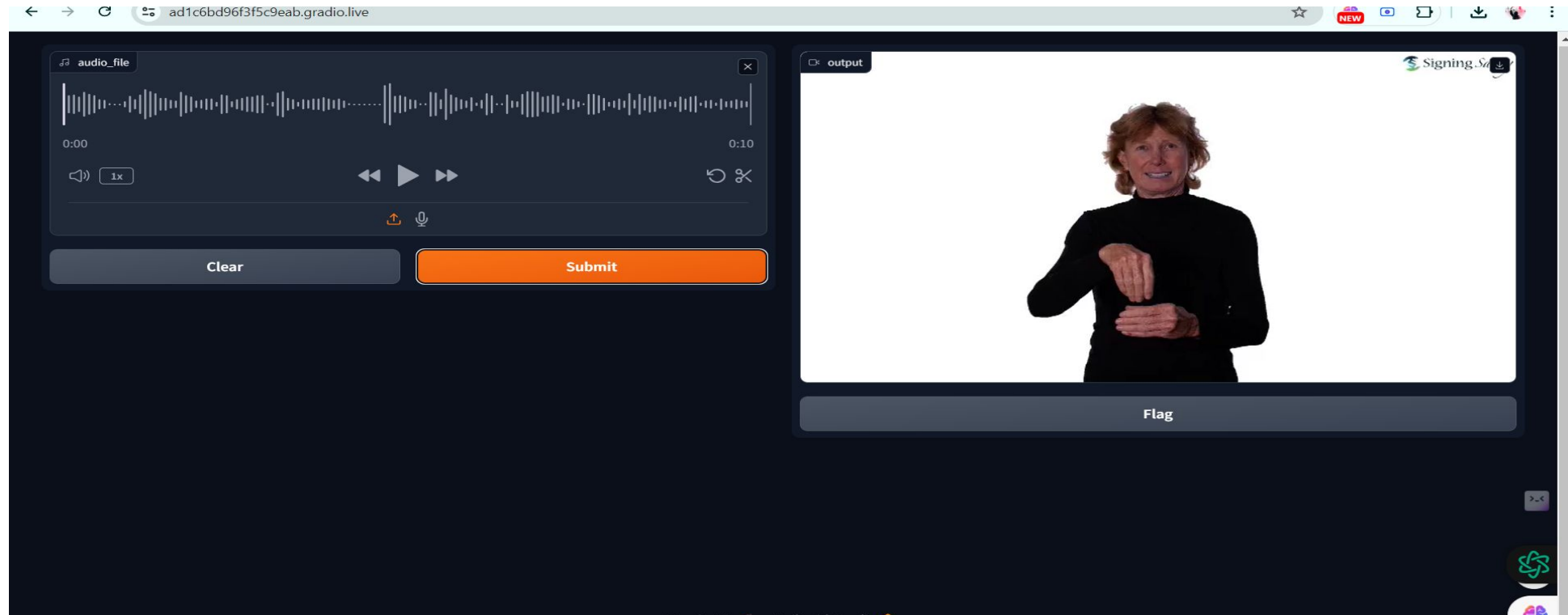
Expansion to Other Languages

The techniques and models developed could be adapted for other sign languages, expanding the system's applicability globally.

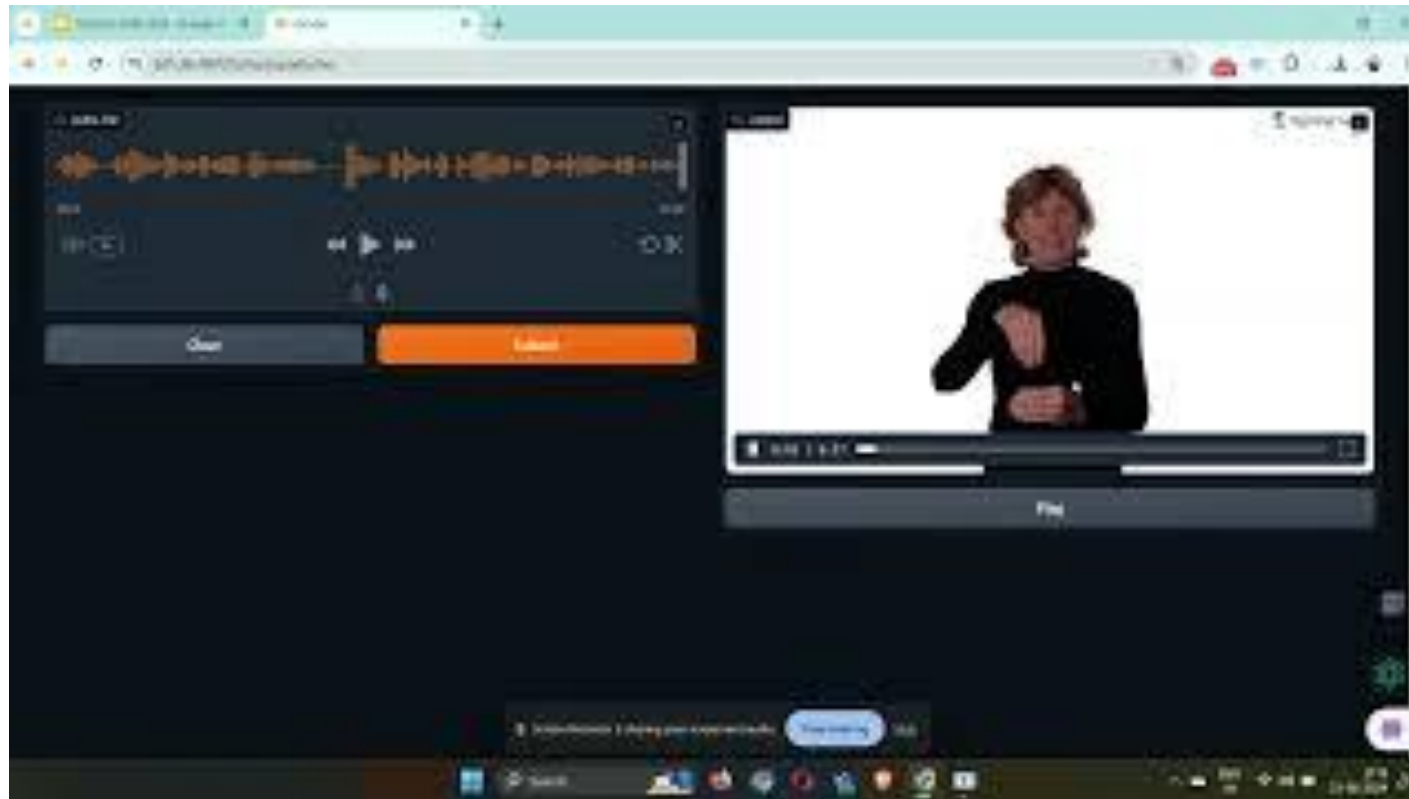
Incorporating Facial Expressions

Future improvements could include the generation of facial expressions, which are a critical components of ASL grammar and meaning.

DEPLOYMENT SCREENSHOT

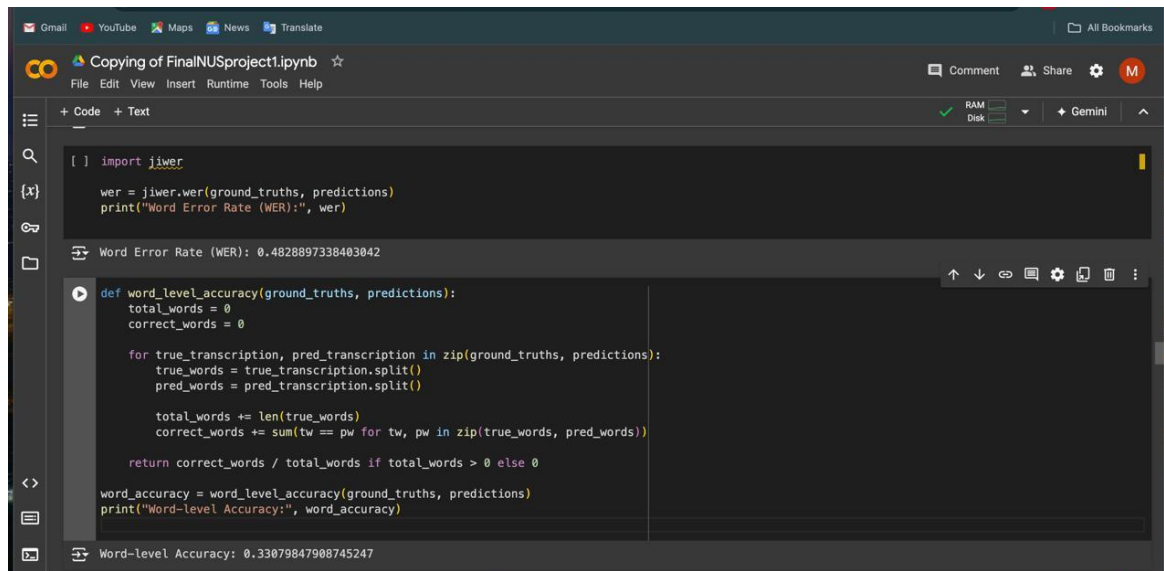


DEPLOYMENT VIDEO



DEPLOYMENT METRICS

Word Level Accuracy



The screenshot shows a Jupyter Notebook interface with a dark theme. The code cell contains the following Python code:

```
[ ] import jiwer

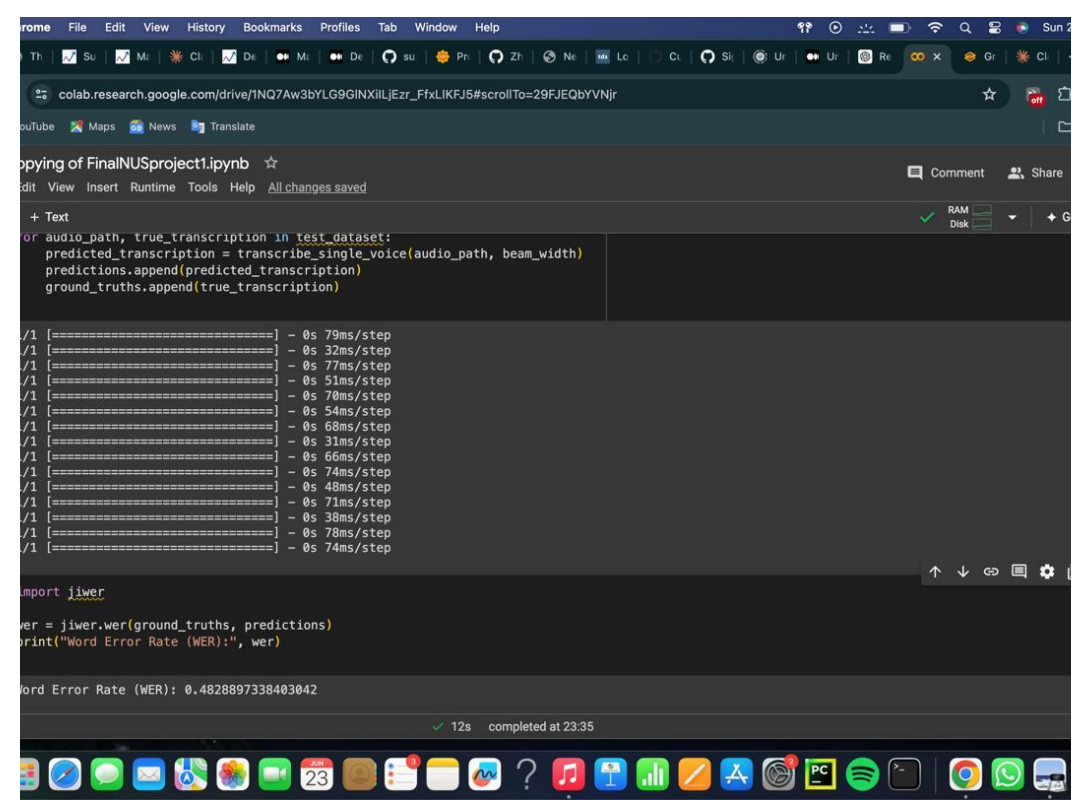
wer = jiwer.wer(ground_truths, predictions)
print("Word Error Rate (WER):", wer)

def word_level_accuracy(ground_truths, predictions):
    total_words = 0
    correct_words = 0
    for true_transcription, pred_transcription in zip(ground_truths, predictions):
        true_words = true_transcription.split()
        pred_words = pred_transcription.split()
        total_words += len(true_words)
        correct_words += sum(tw == pw for tw, pw in zip(true_words, pred_words))
    return correct_words / total_words if total_words > 0 else 0

word_accuracy = word_level_accuracy(ground_truths, predictions)
print("Word-level Accuracy:", word_accuracy)
```

The output cell shows the result: "Word-level Accuracy: 0.33079847908745247".

Word Error Rate(wer)



The screenshot shows a Jupyter Notebook interface with a dark theme. The code cell contains the following Python code:

```
for audio_path, true_transcription in test_dataset:
    predicted_transcription = transcribe_single_voice(audio_path, beam_width)
    predictions.append(predicted_transcription)
    ground_truths.append(true_transcription)
```

The output cell shows a progress bar with the following text:

```
/1 [=====] - 0s 79ms/step
/1 [=====] - 0s 32ms/step
/1 [=====] - 0s 77ms/step
/1 [=====] - 0s 51ms/step
/1 [=====] - 0s 70ms/step
/1 [=====] - 0s 54ms/step
/1 [=====] - 0s 68ms/step
/1 [=====] - 0s 31ms/step
/1 [=====] - 0s 66ms/step
/1 [=====] - 0s 74ms/step
/1 [=====] - 0s 48ms/step
/1 [=====] - 0s 71ms/step
/1 [=====] - 0s 38ms/step
/1 [=====] - 0s 78ms/step
/1 [=====] - 0s 74ms/step
```

The output cell shows the result: "Word Error Rate (WER): 0.4828897338403042".

THANK YOU